

Flujo de compra

Carrito, Stripe, stock, pedidos y emails

Resumen general:

1. El usuario anade productos al carrito.
2. El carrito guarda una intencion de compra, no reserva stock.
3. Al pagar, el backend crea una sesion de Stripe Checkout.
4. Stripe cobra al usuario y avisa al backend.
5. El backend comprueba el pago y revisa el stock real.
6. Si hay stock, descuenta unidades, crea el pedido y vacia el carrito.
7. Finalmente envia el email de confirmacion del pedido.

Idea clave: la compra no queda cerrada al anadir al carrito.

La compra queda cerrada cuando Stripe confirma el pago y el backend crea el pedido.

Carrito y stock

Que ocurre antes de pagar

Frontend:

- El usuario elige producto, talla y cantidad.
- Angular llama al servicio de carrito.

Backend: `CartService.add(...)`

- Busca el producto y comprueba que sea comprable.
- Normaliza la talla: por ejemplo XL, 42, etc.
- Mira el stock de esa talla y limita la cantidad maxima.
- Guarda o actualiza la linea en `cart_items`.

Tabla `cart_items`:

- `user_id`: propietario del carrito.
- `product_id`: producto guardado.
- `size`: talla seleccionada.
- `quantity`: cantidad elegida.

Importante: aqui no se descuenta stock real. Dos usuarios pueden tener la misma ultima unidad en sus carritos. Gana quien complete el pago primero.

Pago con Stripe

Sesion Checkout y control anti-duplicados

Cuando el usuario pulsa "Proceder con el pago":

1. `PaymentService.createCheckoutSession(...)` obtiene el usuario actual.
2. Lee los productos de `cart_items`.
3. Crea `line_items` para Stripe con nombre, precio y cantidad.
4. Crea una sesion Stripe Checkout y devuelve la URL al frontend.
5. Guarda la sesion en `stripe_checkout_sessions` como PENDING.

Estados de `stripe_checkout_sessions`:

- PENDING: sesion creada, todavia sin procesar.
- PROCESSING: el backend esta creando el pedido.
- COMPLETED: pedido creado correctamente.
- FAILED: fallo durante el procesamiento.

Esta tabla es importante porque Stripe puede avisar por webhook y tambien puede volver el usuario desde la pagina de exito. Si llegan dos avisos, COMPLETED evita que se cree el mismo pedido dos veces.

Pedido real

Confirmacion, descuento de stock y limpieza del carrito

Stripe confirma el pago por webhook o por la vuelta del usuario a la app.

`PaymentService.fulfillPaidCheckout(...)` comprueba que `paymentStatus` sea `paid`.

Despues llama a `CartService.checkoutPaidCartForUser(userId)`.

Ese metodo hace el checkout real:

1. Lee el carrito actual del usuario.
2. Por cada item vuelve a cargar el producto desde base de datos.
3. Calcula el stock real de la talla.
4. Si ya no hay stock suficiente, lanza error y no crea pedido.
5. Si hay stock, descuenta unidades en `products.size_stock_json` y `products.stock`.
6. Crea la cabecera en `orders`.
7. Crea las lineas en `order_items`.
8. Vacía `cart_items` del usuario.

Por eso el stock se decide al pagar, no al meter el producto al carrito.

Emails y puntos clave

Confirmacion, marketing y mejoras posibles

Email de pedido:

- Tras crear el pedido se llama a `MarketingEmailService.sendOrderConfirmationEmail(order)`.
- El correo se genera en HTML y se envia con `JavaMailSender`.
- Incluye resumen de productos, cantidades, tallas y total pagado.

Emails de marketing:

- Bienvenida al registrarse.
- Producto anunciado como proximamente.
- Producto lanzado desde proximamente.
- Reposicion de stock.
- Se envian en segundo plano con `@Async` para no bloquear al usuario.

Punto delicado:

- Si dos pagos de la ultima unidad llegan exactamente a la vez, lo ideal seria bloquear la fila del producto o hacer un update atomico de stock.
- Para una tienda pequena el flujo actual es entendible, pero en produccion conviene reforzarlo con una operacion atomica o `SELECT ... FOR UPDATE`.